

Logic Synthesis and Verification

Programming Assignment 1 Report

Yu-Chen Liu

B14901091

September 21, 2026

1 Using ABC

The `show` command did not work on my laptop¹, so I instead outputted as `.dot` file and rendered it with `xdot` software externally. The commands were executed as shown below.

```
liu@Lius-MacBook-Air pa1 % ~/Documents/abc/abc
UC Berkeley, ABC 1.01 (compiled Sep 15 2026 09:40:48)
===== Command history =====
abc 01> read mul.blif
abc 02> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 8 edge = 16 cube = 10 lev = 3
abc 02> write_dot mul.dot
abc 02> strash
abc 03> print_stats
mul          : i/o = 4/ 4 lat = 0 and = 12 lev = 4
abc 03> write_dot mul_strash.dot
abc 03> collapse
abc 04> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 4 edge = 14 bdd = 14 lev = 1
abc 04> write_dot mul_collapse.dot
abc 04> quit
liu@Lius-MacBook-Air pa1 %
```

Figure 1: The screenshot of terminal

The results were then rendered.

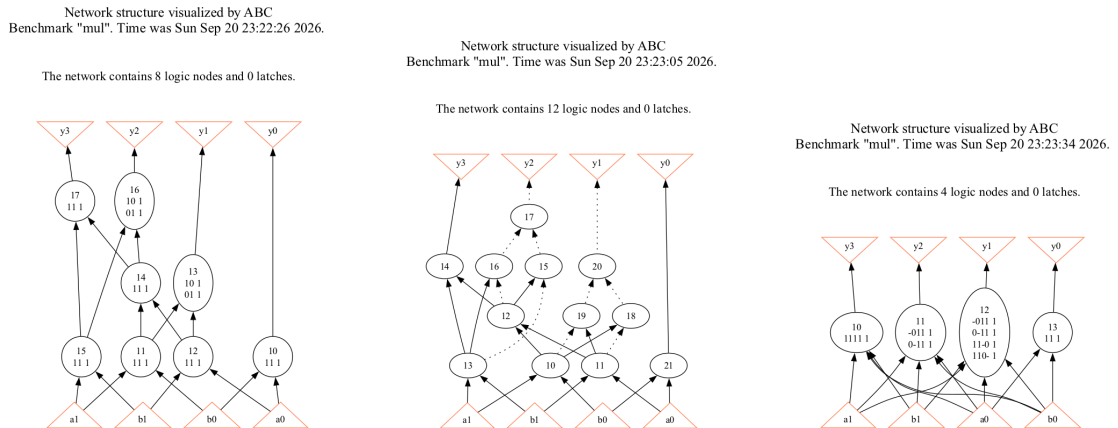


Figure 2: The rendered result (left: original, middle: strashed, right: collapsed)

¹It kept throwing me `sh: gv: command not found` error message. I think it was an environmental issue.

2 ABC Boolean Function Representations

2.1 Comparison

The following shows the executed sequence of command.

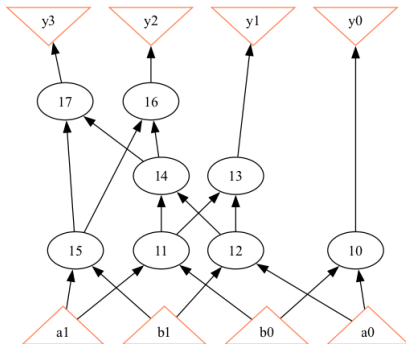
```
liu@Lius-MacBook-Air pa1 % ~/Documents/abc/abc
UC Berkeley, ABC 1.01 (compiled Sep 15 2026 09:40:48)
===== Command history =====
source -s abc.rc
read mul.blif
print_stats
write_dot mul.dot
strash
write_dot mul_strash.dot
collapse
write_dot mul_collapse.dot
=====
abc 01> read mul.blif
abc 02> aig
abc 02> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 8 edge = 16 aig = 12 lev = 3
abc 02> write_dot mul_aig.dot
abc 02> bdd
abc 02> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 8 edge = 16 bdd = 16 lev = 3
abc 02> write_dot mul_bdd.dot
abc 02> quit
```

Figure 3: The screenshot of terminal

The difference can then be compared.

Network structure visualized by ABC
Benchmark "mul". Time was Mon Sep 21 09:49:41 2026.

The network contains 8 logic nodes and 0 latches.



Network structure visualized by ABC
Benchmark "mul". Time was Sun Sep 20 23:23:05 2026.

The network contains 12 logic nodes and 0 latches.

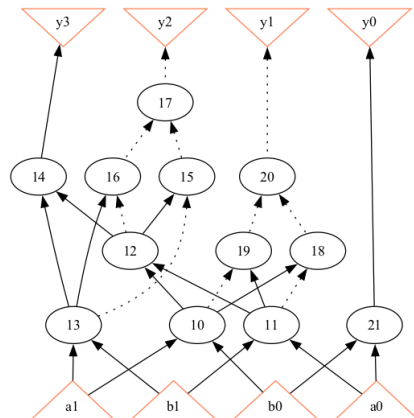
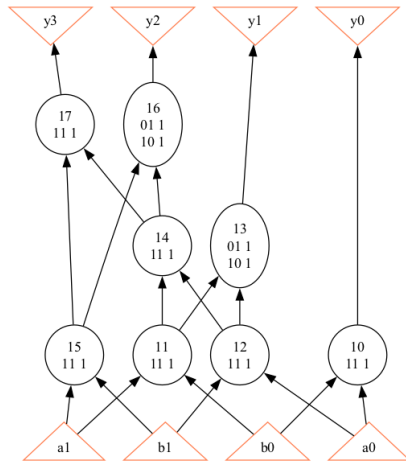


Figure 4: Multiplier in AIG (left) vs. strashed (right) form

The structure in AIG format is the same as the original. The `aig` command simply changes the module to aig format in ABC implicitly. We can see this when `print_stats` is executed. Meanwhile, `strash` command changes the structure, making each node a two-input single AND-inverter gate.

Network structure visualized by ABC
Benchmark "mul". Time was Mon Sep 21 09:50:23 2026.

The network contains 8 logic nodes and 0 latches.



Network structure visualized by ABC
Benchmark "mul". Time was Sun Sep 20 23:23:34 2026.

The network contains 4 logic nodes and 0 latches.

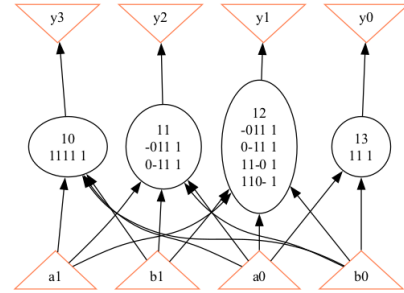


Figure 5: Multiplier in BDD (left) vs. collapsed (right) form

Similarly, the **BDD** command only changes the format of each node to binary-decision representation, without changing the structure. Whereas, the **collapse** command flattens the structure into 1-level circuit, with each node equivalent to a binary-decision tree for each primary output.

2.2 Command Sequence

The sequence of command is: **logic** (convert to a logic network) → **sop** (convert the representation for each node to SOP expression).

```
liu@Lius-MacBook-Air pa1 % ~/Documents/abc/abc
UC Berkeley, ABC 1.01 (compiled Sep 15 2026 09:40:48)
===== Command history =====
source -s abc.rc
read mul.blif
print_stats
write_dot mul.dot
strash
write_dot mul_strash.dot
collapse
write_dot mul_collapse.dot
write_dot mul_aig.dot
write_dot mul_bdd.dot
=====
abc 01> read mul.blif
abc 02> strash
abc 03> print_stats
mul          : i/o = 4/ 4 lat = 0 and = 12 lev = 4
abc 03> logic
abc 04> sop
abc 04> print_stats
mul          : i/o = 4/ 4 lat = 0 nd = 12 edge = 24 cube = 12 lev = 4
abc 04> write_dot mul_sop.dot
abc 04> quit
```

Figure 6: The command sequence for transforming a strashed module to a SOP-represented one

The result is shown below.

Network structure visualized by ABC
Benchmark "mul". Time was Mon Sep 21 11:28:39 2026.

Network structure visualized by ABC
Benchmark "mul". Time was Sun Sep 20 23:23:05 2026.

The network contains 12 logic nodes and 0 latches.

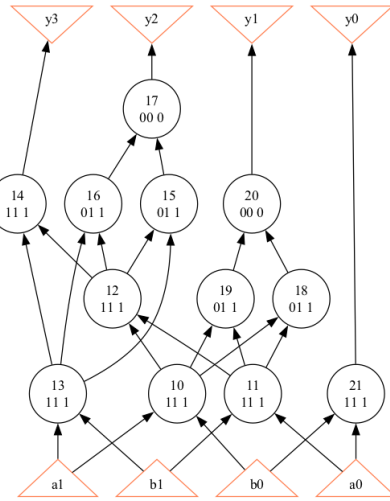
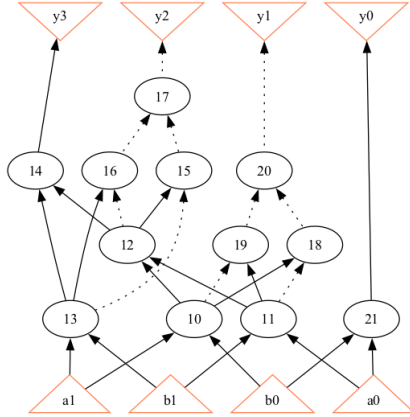


Figure 7: Before (left) and after (right) the transformation

We can see that each node is represented by a SOP expression with single product term. The output of each node is then determined to be complemented or not for inputs of next nodes.